

Детекција на коловозни ленти

Предмет: Дигитално процесирање на слика

Професор: Ивица Димитровски

Автори: Дамјан Зимбаков, Ефимија Цунева

септември 2025

Репозиториум: <https://github.com/zimbakovtech/LaneDetectionCV>

Содржина

Резиме	1
Номенклатура и кратенки	2
1 Вовед	3
1.1 Мотивација	3
1.2 Стратегии	3
2 Дефинирање на проблемот и цели	4
2.1 Дефинирање на проблемот	4
2.2 Цели	4
2.3 Претпоставки	4
2.4 Критериуми за успех	4
3 Преглед на постоечка работа и позадина	5
3.1 Класична обработка на слики	5
3.2 Невронски пристапи	5
3.3 Трговски компромиси	5
4 Преглед на системот	6
4.1 Случаи на употреба	6
4.2 Архитектура на софтверот	6
5 Податоци и претпроцесирање	7
5.1 Извори на податоци	7
5.2 Сплитови и анотации	7
5.3 Чекори за претпроцесирање	7
6 Детали на имплементацијата	8
6.1 OpenCV Pipeline	8
6.1.1 Фајлови и entry points	8
6.1.2 Алгоритам	8
6.1.3 Псевдо-код	8
6.2 U-Net Pipeline	9
6.2.1 Фајлови и entry points	9
6.2.2 Сумирање на моделот	9
6.2.3 Функција на губење и оптимизација	9
6.2.4 Инференција и пост-процесирање	9
6.3 Хиперпараметри и конфигурација (пример)	9

7	Метрики за евалуација	10
7.1	Формални дефиниции	10
7.2	Протокол за евалуација	10
8	Архитектура и структура на системот	11
8.1	Работна околина и репродуктивност	11
9	Инсталација и подесување	12
9.1	Зависности	12
9.2	Команди за поставување на околина	12
9.3	Белешки за GPU	12
10	Примери за употреба	13
10.1	OpenCV pipeline	13
10.2	U-Net pipeline	13
10.3	OpenCV vs U-Net: споредба на FPS	13
11	Експерименти и резултати	14
11.1	Квантитативни резултати	14
11.2	Квалитативни резултати	14
11.3	Перформанси и сложеност	14
12	Дискусија	15
12.1	Споредба	15
12.2	Сценарија за неуспех и мерки за ублажување	15
12.3	Ограничувања и закани по валидност	15
13	Потенцијални предизвици и идна работа	16
13.1	Предизвици	16
13.2	Идни подобрувања	16
13.3	Етички и правни аспекти	16
14	Заклучок	17
A	CLI помош и пример конфигурации	18
A.1	Примери на CLI команди	18
A.2	Структура на репозиториум	18
A.3	Лиценца	18

Резиме

Оваа документација прецизно ја опишува системската реализација со два паралелни пристапи за детекција на коловозни ленти, изработена во рамките на проектот по предметот *Дигитално процесирање на слика*. Развивме и споредивме два комплементарни pipeline-a:

- класичен OpenCV pipeline базиран на отстранување на шум, Canny edge detection, маскирање на регион од интерес (ROI), екстракција на сегменти преку Hough и робусно прилагодување на линии со временска импутација, и
- deep-learning pipeline користејќи U-Net архитектура (encoder-decoder) за семантичка сегментација на пиксели кои припаѓаат на ленти, со лесна пост-процесинг визуелизација.

Во текстот детално се дискутираат подготовката на податоци, имплементациските детали, метрики за евалуација (IoU, Precision/Recall, F1, Pixel Accuracy), експерименти, резултати, ограничувања и насоки за идни подобрувања.

Клучни зборови: Lane detection; semantic segmentation; OpenCV; U-Net; Canny; Hough transform; Grayscale; Computer Vision;

Номенклатура и кратенки

- **ROI** — Region of Interest (регион од интерес)
- **IoU** — Intersection over Union
- **FPS** — Frames Per Second
- **BCE** — Binary Cross-Entropy
- **TP/FP/FN/TN** — True/False Positive/Negative

1. Вовед

1.1 Мотивација

Детекција на коловозните ленти е фундаментална задача за перцепција во системи за помош при возење и автономна возачка технологија. Сигурната детекција на ленти овозможува латерална контрола, предупредување за напуштање на лентата и планирање на патеката. Традиционалните методи од Computer Vision остануваат популарни поради нивната едноставност и ниска цена на компјутерска моќ, но се чувствителни на осветлување и деградација на ознаките. Deep learning пристапите обезбедуваат поголема прецизност и робустност во реални сценарија, но бараат анотирани податоци и повеќе пресметковни ресурси.

1.2 Стратегии

Овој проект ги имплементира и споредува двете стратегии за детекција на ленти (OpenCV и U-Net). Проектот вклучува OpenCV pipeline, U-Net pipeline имплементиран со Keras/TensorFlow, скрипти за подготовка на dataset/тренинг/инференција и алатки за евалуација. Двете стратегии се:

- Едноставна, читлива имплементација на класичен OpenCV pipeline со Grayscale, Blur, ROI, Canny, Hough и робусно прилагодување на линии со временска импутација.
- U-Net pipeline со скрипти за подготовка на податоци, тренирање, инференција и евалуација на маски; поддршка за снимен модел.

2. Дефинирање на проблемот и цели

2.1 Дефинирање на проблемот

Детекција и визуелизација на коловозни ленти во предно насочен видео запис (forward-facing) сниман со една камера поставена близу ветробранското стакло. Лентите можат да бидат полни или испрекинати, бели или жолти, и може да бидат делумно покриени, засенчени или деградирани, со променлив квалитет на видео.

2.2 Цели

1. Имплементирање брз (CPU-friendly) OpenCV pipeline за практично (real-time) прикажување на лентите.
2. Имплементирање U-Net pipeline кој ја подобрува робустноста и прецизноста во потешки услови.
3. Обезбедување скрипти за подготовка на податоци, тренирање и евалуација.
4. Квантитативно и квалитативно споредување на двата пристапа.

2.3 Претпоставки

- Една предно насочена камера, умерен наклон и ограничена дисторзија на леќата.
- Две примарни граници на лентите (лево и десно) често видливи во регион со облик на пат, односно трапез (ROI).
- Типични влезни резолуции во опсегот 720p–1080p; големината на влезот може да се промени во самиот pipeline.

2.4 Критериуми за успех

Проектот се смета за успешен доколку:

- OpenCV pipeline обезбедува стабилна визуелизација со FPS во *реално-време* користејќи релативно ниска процесорска моќ, и
- U-Net pipeline постигне мерливо подобрување на IoU/F1 во споредба со OpenCV на истиот сет на слики, користејќи значително повеќе ресурси.

3. Преглед на постоечка работа и позадина

3.1 Класична обработка на слики

Клучни компоненти користени во класичниот pipeline:

- **Canny edge detection** за екстракција на рабови базирана на градиент.
- **Hough transform** за реконструкција на линиски сегменти од edge map-та.
- **Region-of-interest** маскирање за отстранување на неважни области (небото, хауба, позадина).
- Полиноминално прилагодување или линеарно прилагодување за добивање визуелни ленти по страна.

3.2 Невронски пристапи

Learning pipeline-от е базиран на U-Net архитектурата. Други lane-specific модели (на пр. SCNN, ENet-SAD) воведуваат структурни промени и механизми за внимание кои се релевантни за идни подобрувања.

3.3 Трговски компромиси

- **OpenCV**: лесно разбирлив, ефикасен, бара малку податоци; чувствителен на осветлување и истрошени ознаки.
- **U-Net**: прецизен и подобро се генерализира со достапни анотирани податоци; бара повисоки компјутерски ресурси и внимателно тренирање.

4. Преглед на системот

Системот работи со два независни pipeline-a:

- **OpenCV pipeline:** Preprocess → Grayscale → Gaussian Blur → Canny edges → ROI → Hough segments → групирање по slope (лево/десно) → fit lines → временска импутација → overlay.
- **U-Net pipeline:** Preprocess → U-Net predict mask → опционално морфолошко чистење → overlay и export.

4.1 Случаи на употреба

Користете OpenCV на системи со ограничени ресурси, а U-Net кога има достапни анотирани модели и се бара поголема прецизност.

4.2 Архитектура на софтверот

Pipeline-ите се раздвоени во директориуми `src/opencv_pipeline` и `src/u_net_pipeline`. Секој има јасни entry points (`main.py`) и помошни модули. Резултатите (видеа, слики, маски) се чуваат под `results/`. Влезните видеа се под `data/raw/`.

5. Податоци и претпроцесирање

5.1 Извори на податоци

Влезните видеа се сместени во `data/raw/`: `road_video_1.mp4 ... road_video_5.mp4`. Опционално, може да се интегрираат јавни датасетови како TuSimple, CULane и BDD100K.

5.2 Сплитови и анотации

Парови за тренирање и валидација се чувани под `data/processed/`. За U-Net целите се бинарни lane-mask слики. Ако ground-truth анотациите недостасуваат, може да се генерираат псевдо-маски преку хевристички правила или со помош на OpenCV pipeline.

5.3 Чекори за претпроцесирање

- Извлечете кадри на фиксен FPS (на пр. 5 FPS).
- Resize на кадрите (пример: ширина=512, висина=256).
- Нормализирајте вредности на пиксели; примени аугментации (brightness/contrast jitter, мали ротации, Gaussian noise).

6. Детали на имплементацијата

6.1 OpenCV Pipeline

6.1.1 Фајлови и entry points

- `src/opencv_pipeline/detect.py` – логика за детекција по кадар.
- `src/opencv_pipeline/main.py` – главен фајл за извршување кој ги извлекува кадрите
- `src/opencv_pipeline/functions/*` – помошни функции: preprocessing, ROI, draw utilities.

6.1.2 Алгоритам

Главни чекори:

1. Конверзија во grayscale.
2. Примена на Gaussian blur (на пр. 5×5).
3. Пуштање на Canny edge detector (настрани поставливи threshold-и).
4. Маскирање со полигона ROI (трапец во форма на пат).
5. Пуштање на Probabilistic Hough Transform за екстракција на сегменти.
6. Филтрирање на сегментите по slope и групирање во left/right.
7. Fit lines во форма $x = a \cdot y + b$ користејќи robust polyfit на краевите на сегментите.
8. Временска импутација/перзистентност за поврзување на испрекинати ленти.
9. Рендерирање на overlay и запишување на видео кадри.

6.1.3 Псевдо-код

```
1 frame = read_frame()
2 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
3 blur = cv2.GaussianBlur(gray, (5,5), 0)
4 edges = cv2.Canny(blur, 100, 200)
5 roi = cv2.fillPoly(mask, [vertices], match_mask_color)
6 segments = cv2.HoughLinesP(masked, rho=6, theta=np.pi / 60,
7                               threshold=160, minLineLength=40, maxLineGap=25)
8 left, right = split_by_slope(segments, slope_min=0.5)
9 left_fit = robust_fit_yx(left)      # returns coefficients (a,b)
10 right_fit = robust_fit_yx(right)
11 overlay = draw_lines(frame, [left_fit, right_fit])
12 write_frame(overlay)
```

6.2 U-Net Pipeline

6.2.1 Фајлови и entry points

- `src/u_net_pipeline/main.py` – CLI за `prepare/train/infer`.
- `src/u_net_pipeline/src/model.py` – U-Net дефиниција на моделот.
- `src/u_net_pipeline/scripts/train.py` – training loop и логирање.
- `src/u_net_pipeline/scripts/inference.py` – infer кон маски/overlay.

6.2.2 Сумирање на моделот

U-Net: encoder–decoder со skip connections. Типичен влезен размер: 256×512 (HxW). Финален слој: 1×1 conv + sigmoid кој враќа per-pixel probability map.

6.2.3 Функција на губење и оптимизација

Чести избори:

- Binary cross-entropy (BCE) или BCE + Dice loss за справување со небаланс на класи.
- Optimizer: Adam со learning rate во опсегот $[1e-4, 1e-3]$.
- Early stopping и checkpointing врз валидациски IoU.

6.2.4 Инференција и пост-процесирање

- Предвидување на probability map; threshold (на пр. 0.5) за бинарна маска.
- Опционално морфолошко opening/closing за отстранување на мали blob-ови.
- Overlay на маската (alpha blending) врз оригиналните кадри и запишување на излезно видео.

6.3 Хиперпараметри и конфигурација (пример)

Табела 6.1: Клучни хиперпараметри за U-Net тренинг (пример вредности).

орrule	Параметар	Вредност
	Input size	256×512 (H x W)
	Batch size	4
	Epochs	10–20
	Optimizer	Adam (LR 10^{-4})
	Loss	BCE + Dice (по избор)
	Augmentation	brightness/contrast, rotations ($\pm 5^\circ$)
	Early stopping	patience 5–10
	Checkpoints	<code>src/u_net_pipeline/models/</code>

7. Метрики за евалуација

Сегментацијата ја евалуираме на ниво на пиксел користејќи:

- Intersection over Union (IoU)
- Precision, Recall, F1-score
- Pixel Accuracy

Имплементацијата на метриците е достапна во `src/u_net_pipeline/scripts/metrics.py` во репозиториумот.

7.1 Формални дефиниции

Нека Y е бинарна ground-truth маска, а $\hat{Y}$ е бинарна предикција. Нека TP, FP, FN, TN се вообичаените броеви на пиксели.

$$\text{IoU} = \frac{|Y \cap \hat{Y}|}{|Y \cup \hat{Y}|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}, \quad (7.1)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (7.2)$$

$$\text{F1} = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (7.3)$$

$$\text{Pixel Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \quad (7.4)$$

7.2 Протокол за евалуација

- Предикциите се извлекуваат за секоја слика во `val` или `test` сплит; метриците се пресметуваат по-слика и се усреднетуваат.
- За фер споредба, OpenCV pipeline генерира бинарна маска со исцртани предвидени линии (фиксна дебелина), а U-Net генерира бинарна сегментациска маска со прагот 0.5.
- Маските се изменуваат (nearest-neighbor) на резолуцијата на ground-truth пред пресметка на метрики.

8. Архитектура и структура на системот

Структура на репозиториумот:

```
data/  
  raw/  
  processed/  
src/  
  opencv_pipeline/  
  u_net_pipeline/  
results/  
requirements.txt  
README.md
```

Клучни entry points:

- OpenCV: `python src/opencv_pipeline/main.py`
- U-Net: `python src/u_net_pipeline/main.py [prepare|train|infer]`

8.1 Работна околина и репродуктивност

- Python: ≥ 3.8 ; TensorFlow/Keras и OpenCV според `requirements.txt`.
- Фиксирање на seed (`-seed 42`) каде е применливо; логирање на верзии на пакети.
- Запишување на *checkpoints* и историја на тренинг во `src/u_net_pipeline/models/`.

9. Инсталација и подесување

9.1 Зависности

- Python $\geq$ 3.8
- Пакети наведени во `requirements.txt` (инсталирајте со `pip install -r requirements.txt`)
- Опционално: GPU-верзија на TensorFlow (со соодветни CUDA/cuDNN верзии)

9.2 Команди за поставување на околина

```
1 python -m venv venv
2 # Windows PowerShell:
3 ./venv/Scripts/Activate.ps1
4 pip install -r requirements.txt
```

9.3 Белешки за GPU

При користење на GPU-верзија на TensorFlow, потребно е усогласување на верзиите на CUDA/cuDNN со верзијата на TensorFlow. Во спротивно користете CPU build.

10. Примери за употреба

10.1 OpenCV pipeline

```
1 # Process all videos in data/raw and save outputs to results/  
2 python src/opencv_pipeline/main.py
```

10.2 U-Net pipeline

```
1 # Prepare dataset  
2 python src/u_net_pipeline/main.py prepare --fps 5 --img-width 512  
   --img-height 256  
3  
4 # Train model  
5 python src/u_net_pipeline/main.py train --epochs 10 --batch-size 4  
   --model-out src/u_net_pipeline/models/best_model.h5  
6  
7 # Inference on a video  
8 python src/u_net_pipeline/main.py infer \  
9   --input data/raw/road_video_5.mp4 \  
10  --model src/u\_net\_pipeline/models/best_model.h5 \  
11  --output results/output_video.mp4
```

10.3 OpenCV vs U-Net: споредба на FPS

Времето на извршување зависи од резолуцијата и хардверот. OpenCV обично надминува реално-временски FPS на CPU за 720p; U-Net зависи од моделската големина и достапност на GPU.

11. Експерименти и резултати

11.1 Квантитативни резултати

Табела 11.1: Примерен резиме на експерименталните резултати.

Method	IoU	PixelAcc	Precision	Recall	F1
OpenCV	0.52	0.95	0.61	0.52	0.54
U-Net	0.68	0.94	0.77	0.73	0.79

11.2 Квалитативни резултати



(a) OpenCV pipeline output



(б) U-Net mask overlay

Слика 11.1: Пример страна-до-страна: OpenCV vs U-Net квалитативни резултати.



Слика 11.2: Визуелизација на сегментациска маска и линии на пример кадар.

11.3 Перформанси и сложеност

- **OpenCV:** Canny и Hough имаат приближно линеарна сложеност во број пиксели и број детектирани сегменти; вкупно близу $\mathcal{O}(HW)$ по кадар.
- **U-Net:** сложеноста е пропорционална на бројот на параметри и конволутивни операции; приближно $\mathcal{O}(HW \cdot C)$, каде C зависи од ширината и длабочината на моделот.

12. Дискусија

12.1 Споредба

OpenCV е брз и интерпретабилен, но има проблеми со сенки, кривини, низок квалитет на видео и испрекинати линии. U-Net подобро се генерализира кога има доволно анотирани податоци, но користи повисоки пресметковни и анотациски ресурси.

12.2 Сценарија за неуспех и мерки за ублажување

- **Испрекинати линии:** спојување средни вредности помеѓу линии и импутација; зголемени Hough gap вредности.
- **Кривини на коловозот:** тренирање полиномни кривини или користење segment-wise RANSAC; разгледување на IPM и spline fitting.
- **Domain shift:** фино-подесување (fine-tune) на target податоците; data augmentation и domain adaptation.

12.3 Ограничувања и закани по валидност

- Ограничена големина и разновидност на податоци може да води до пристрасност на метриците.
- Различни резолуции и FOV меѓу сетовите може да нарушат споредливост.
- Недостиг на GT маски води до процена врз псевдо-етикети; бројките треба да се интерпретираат внимателно.

13. Потенцијални предизвици и идна работа

13.1 Предизвици

Репродуктивност, пристрасност во податоците (dataset bias), шум во анотациите, ограничувања при објавување и overfitting.

13.2 Идни подобрувања

- Проценка на мулти-ленти (повеќе од една) и нумерирање на ленти.
- Временско филтрирање и следење (Kalman filters, optical flow).
- Inverse perspective mapping (IPM) со bird's-eye проекција и splines.
- Квантизација на моделот за вградување во embedded уреди.

13.3 Етички и правни аспекти

Користење јавни датасетови и видео материјали мора да биде во согласност со нивните лиценци и приватносни политики; проектот е објавен под MIT лиценца.

14. Заклучок

Со овој проект покажавме како две различни парадигми за детекција на коловозни ленти можат да коегзистираат и да се надополнуваат: класичниот, детерминистички и лесно интерпретабилен OpenCV pipeline, и U-Net пристапот за семантичка сегментација. Репозиториумот испорачува целосна инфраструктура: скрипти за подготовка на податоци, тренинг и инференција, преглед во реално време на OpenCV излез и визуелизација на лентите. Тоа е најбитниот дел на системот, не само „да работи“, туку и да биде мерлив, споредлив и визуелен.

Емпириски, U-Net доследно го надминува OpenCV во сегментациските метрики што навистина го мерат квалитетот на детекцијата на лентите, особено во посложени услови (прекини, засенчувања, деградација на ознаките, кривини). Наспроти тоа, OpenCV задржува предност во брзина/FPS и едноставност на подесување, што го прави одлична опција за системи со ограничени ресурси и како брз прототип. Важно е да се нагласи дека Pixel Accuracy, иако понекогаш е повисока кај OpenCV, е помалку информативна поради силниот небаланс во класите (доминира позадината). Затоа IoU/F1 се порелевантни критериуми при споредба на двата пристапи.

Критичните заклучоци се јасни:

- Ако приоритет се ниски ресурси, минимална зависност од анотации и брзо разбирање, и најбитно, излез во реално време, класичниот OpenCV pipeline е подобриот избор.
- Ако приоритет е робустност и прецизност во реални, динамични услови, со достапни етикетиран податоци и GPU, U-Net е супериорен.
- Најпрактичниот пат кон продукција е хибридно спојување на двата пристапи: OpenCV да обезбеди геометриски приори и ROI, U-Net да ја реши сегментацијата, а пост-процесирањето да воведо временско стабилизирање.

Од инженерска перспектива, проектот поставува стабилна основа за надградба: модуларна архитектура, јасни точки на проширување и мерлива евалуација. За да се премине од прототип кон подготвен систем, препорачуваме: проширување на датасетот (ноќни/влажни услови), фино-подесување (fine-tuning) на моделот на таргетиран домен, како и додавање времески модели.

Заклучно, проектот ја поместува работата од „демо“ кон мерлив истражувачки излез на видео, со јасни компромиси и насоки за избор меѓу две опции. OpenCV обезбедува брзина и транспарентност; U-Net обезбедува точност и робустност. Информираната комбинација од двете, поткрепена со пристопна евалуација, е најдобрата стратегија за системи наменети за реални ситуации.

A. CLI помош и пример конфигурации

A.1 Примери на CLI команди

```
1 python src/opencv_pipeline/main.py
2
3 # The U-Net code and scripts are under 'src/u_net_pipeline/'
4 python src/u_net_pipeline/main.py prepare --fps 5 --img-width 512
   --img-height 256
5 python src/u_net_pipeline/main.py train --epochs 10 --batch-size 4
   --model-out models/best_model.h5
6 python src/u_net_pipeline/main.py infer --input data/raw/
   road_video_5.mp4 --model models/best_model.h5 --output results/
   output_video.mp4
7
8 python src/u_net_pipeline/main.py -h
```

A.2 Структура на репозиториум

```
data/
  raw/           # Влезни видеа (.mp4)
  processed/     # спремени слики/маски
src/
  opencv_pipeline/
    main.py, detect.py, functions/
  u_net_pipeline/
    main.py, scripts/, src/, models/
results/
  images/, masks/, *.mp4
requirements.txt
README.md
LICENSE
```

A.3 Лиценца

Проектот е лиценциран под MIT; види LICENSE во репозиториумот.